IM – Induction Game

Shahrukh Mohiuddin

Game Concept

Game Master

Your role:
You (the AI) will propose inputs for a hidden function so you can infer it.

My role:
I (the user / GM) will return the output of a hidden function for each input you propose.

Goal:
Infer the exact function:
$**SIGNATURE_OF_CURRENT_FUNCTION**$

Context:
Category: $CATEGORY$.
This function takes NUM_PARAMS parameter(s) with names and types:
$PARAM_LIST$
(Format shown below.)
.
.
.

Output: 0

Player 1

(x: int, y: int) -> int

Context:
Category: math.
This function takes 2 parameter(s) with names and types:
x: int, y: int
(Format shown below.)

Input: 1,1

Input: 2,-3

Updates

- Test cases are now moved to instance.json level
- This allows for manual edition of test cases

```
"test_cases": [
  {
    "args": [
      50,
      -10
    ],
    "expected": 40
  },
  {
    "args": [
      -100,
      -50
    ],
    "expected": -150
  },
  {
    "args": [
      -10,
      0
    ],
    "expected": -10
  },
  {
    "args": [
      -2,
      -99
```

```
"test_cases": [
  {
    "args": [
      "a"
    ],
    "expected": "a"
  },
  {
    "args": [
      "123"
    ],
    "expected": "321"
  },
  {
    "args": [
      " "
    ],
    "expected": " "
  },
  {
    "args": [
      "aba"
    ],
    "expected": "aba"
  },
  {
```

```
"test_cases": [
  {
    "args": [
      "123",
      0
    ],
    "expected": "123"
  },
  {
    "args": [
      "aA",
      99
    ],
    "expected": "vV"
  },
  {
    "args": [
      "ab",
      99
    ],
    "expected": "vw"
  },
  {
    "args": [
      "hello",
      -2~
```

Updates

- Win / Loss is now binary again (Its either a 1 or a 0)
- Bench score 100 = Win
- Bench score 0 = Loss
- This helps in determining if the function was solved or not

Game Verdict: WIN

Accuracy Score: 1.0

Updates

Pre-probe logging

- Input arguments (probe_args)
- Outputs (probe_output)
- Probe round counter
- All observed pairs are stored as observed_pairs

INPUT:1, 2
HYPOTHESES:
- candidate: f_add
- candidate: abs_diff
CONFIDENCE:
- 0.5
- 0.5

{'probe_round': 1,
'probe_args': [1, 2],
'probe_output': 3}

Error logging

- parse_error_count: model violated output format
- runtime_error_count: function execution / evaluation errors

Parse error count: 0

Runtime error count: 0

Updates

- Addition of metrics to answer the question: “Did the guess happen at the right time?”
- Turns used
- Slack (how many turns remain)
- Efficiency (A normalized, early or late stop score)
  - efficiency = 1 - (probe_round - 1) / max(1, (max_turns - 1))

Efficiency Score: 0.25

Slack Score: 1

{'max_allowed_turns': 5, 'total_turns_used': 4, 'total_turns_remaining': 1}

Updates

- Does the final guessed program reproduce the outputs that have already been observed?
- Metrics:
  - internal_consistency_score (0–1)
  - internal_consistency_all_observed (bool)
  - internal_consistency_violations (count)
- The idea is to see if there are any guesses that contradict with the evidence that has been gathered during gameplay

```
{'internal_consistency_score': 1.0, 'internal_consistency_all_observed': True,
'internal_consistency_violations': 0, 'observed_pair_count': 4}
```

Updates

{'format_errors': 0, 'top1_hit_at_stop': 1, 'top3_hit_at_stop': 1, 'prob_true_at_stop': 0.99, 'avg_neg_log_prob_true': 0.27779143868493245, 'top1_switches': 0}

- Tracking hypothesis with Candid
  - hypothesis_prob_true_at_stop (did it “know it knew”?)
  - hypothesis_top1_hit_at_stop (If there are a bunch of functions, does the true function get the highest confidence or not?)
  - hypothesis_top3_hit_at_stop (Was the true function at least in the top 3?)
  - hypothesis_avg_negative_log_prob_true (log-loss over turns: speed of convergence)
  - hypothesis_top1_switches (thrashing vs stable convergence)
  - hypothesis_format_error_count (did it follow the protocol)
- What we end up measuring: Belief updating upon interaction

Updates

- Defining a candidate pool
  - Candidate pool = functions with same signature + same category + same difficulty
  - This keeps hypotheses comparable and prevents trivial candidates (wrong types) or huge pools.

INPUT:8, 0
HYPOTHESES:
- candidate: f_add
- candidate: abs_diff
CONFIDENCE:
- 0.99
- 0.01

INPUT:1, 2
HYPOTHESES:
- candidate: f_add
- candidate: abs_diff
CONFIDENCE:
- 0.5
- 0.5

INPUT:5, 7
HYPOTHESES:
- candidate: f_add
- candidate: abs_diff
CONFIDENCE:
- 0.7
- 0.3

INPUT:10, 3
HYPOTHESES:
- candidate: f_add
- candidate: abs_diff
CONFIDENCE:
- 0.95
- 0.05

Updates

- Did the model ask inputs that produced **new information**, or did it keep getting the same output?
  - **probe_count**
  - **unique_input_count** and **input_diversity** = **unique_input_count / probe_count**
  - **unique_output_count** and **novelty_rate** = **unique_output_count / probe_count**
  - optional output_entropy_bits:
    - 0.0 bits: all outputs identical (very low information from probing).
    - ~1.0 bit: roughly two outputs equally common.
    - ~2.0 bits: roughly four outputs equally common.
    - In general: higher = more distinct outputs and/or more balanced variety.

Probes: 4 | Unique inputs: 4 (diversity=1.00) | Unique outputs: 4 (novelty=1.00, entropy=2.00 bits)

Finally

GUESS:
```python
def solution(x, y):
return x + y
```
The function returns the sum of x and y.

{'internal_consistency_score': 1.0, 'internal_consistency_all_observed': True, 'internal_consistency_violations': 0, 'observed_pair_count': 4}

Accuracy Score: 1.0

Efficiency Score: 0.25

Slack Score: 1

Game Verdict: WIN

That is correct!
The hidden function was:

```
def f_add(x: int, y: int) -> int:
return x + y
```

Your guess:

```
def solution(x, y):
return x + y
```

{'max_allowed_turns': 5, 'total_turns_used': 4, 'total_turns_remaining': 1}

Parse error count: 0

Finally

Your guess:

```
def solution(x, y):
return x + y
```

{'max_allowed_turns': 5, 'total_turns_used': 4, 'total_turns_remaining': 1}

Parse error count: 0

Runtime error count: 0

{'format_errors': 0, 'top1_hit_at_stop': 1, 'top3_hit_at_stop': 1, 'prob_true_at_stop': 0.99, 'avg_neg_log_prob_true': 0.27779143868493245, 'top1_switches': 0}

Probes: 4 | Unique inputs: 4 (diversity=1.00) | Unique outputs: 4 (novelty=1.00, entropy=2.00 bits)

Some examples

That is correct!
The hidden function was:

```
def caesar_shift(s: str, k: int) -> str:
"""
Shift letters by k (Caesar cipher).
Only shifts A-Z and a-z; other chars unchanged.
"""
out = []
k = k % 26
for ch in s:
if "a" <= ch <= "z":
out.append(chr((ord(ch) - ord("a") + k) % 26 + ord("a")))
elif "A" <= ch <= "Z":
out.append(chr((ord(ch) - ord("A") + k) % 26 + ord("A")))
else:
out.append(ch)
return "".join(out)
```

Your guess:

```
def solution(s: str, k: int) -> str:
res = []
for c in s:
if 'a' <= c <= 'z':
res.append(chr((ord(c) - ord('a') + k) % 26 + ord('a')))
elif 'A' <= c <= 'Z':
res.append(chr((ord(c) - ord('A') + k) % 26 + ord('A')))
else:
res.append(c)
return ''.join(res)
```

'internal_consistency_violations': 0, 'observed_pair_count': 7}

Accuracy Score: 1.0

Efficiency Score: 0.33333333333333337

Slack Score: 3

Game Verdict: WIN

That is correct!
The hidden function was:

```
def rotate_left(xs: List[int], k: int) -> List[int]:
"""Rotate list left by k positions."""
if not xs:
return []
n = len(xs)
k = k % n
return xs[k:] + xs[:k]
```

Your guess:

```
def solution(xs, k):
n = len(xs)
if n == 0:
return []
k = k % n
return xs[k:] + xs[:k]
```

{'max_allowed_turns': 10, 'total_turns_used': 7, 'total_turns_remaining': 3}

Parse error count: 0

Runtime error count: 0

That is correct!
The hidden function was:

```
def edit_distance(a: str, b: str) -> int:
"""
Levenshtein edit distance (insert/delete/replace), O(len(a)*len(b)).
"""
n, m = len(a), len(b)
dp = list(range(m + 1))
for i in range(1, n + 1):
prev_diag = dp[0]
dp[0] = i
for j in range(1, m + 1):
temp = dp[j]
cost = 0 if a[i - 1] == b[j - 1] else 1
dp[j] = min(
dp[j] + 1, # delete
dp[j - 1] + 1, # insert
prev_diag + cost # replace
)
prev_diag = temp
return dp[m]
```

Your guess:

```
def solution(a, b):
m, n = len(a), len(b)
dp = [[0]*(n+1) for _ in range(m+1)]
for i in range(m+1):
dp[i][0] = i
for j in range(n+1):
dp[0][j] = j
for i in range(1, m+1):
for j in range(1, n+1):
if a[i-1] == b[j-1]:
dp[i][j] = dp[i-1][j-1]
else:
dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])
return dp[m][n]
```

{'max_allowed_turns': 15, 'total_turns_used': 13, 'total_turns_remaining': 2}

Slack Score: 2

Game Verdict: LOSS

Incorrect
. Failed Cases:
Input: ['([)]'] | Expected: False | Got: True

Hidden function:

```
def balanced_parentheses(s: str) -> bool:
"""
Check if (), [], {} are balanced and properly nested.
Ignores non-bracket characters.
"""
stack = []
opening = {"(": ")", "[": "]", "{": "}"}
closing = {")", "]", "}"}
for ch in s:
if ch in opening:
stack.append(opening[ch])
elif ch in closing:
if not stack or stack.pop() != ch:
return False
return len(stack) == 0
```

Your guess:

```
def solution(s: str) -> bool:
count = 0
for c in s:
if c == '(':
count += 1
elif c == ')':
count -= 1
if count < 0:
return False
return count == 0
```

{'max_allowed_turns': 15, 'total_turns_used': 13, 'total_turns_remaining': 2}

{'internal_consistency_score': 1.0, 'internal_consistency_all_observed': True, 'internal_consistency_violations': 0, 'observed_pair_count': 8}

Accuracy Score: 0.7

Efficiency Score: 0.2222222222222222

Slack Score: 2

Game Verdict: LOSS

Incorrect
. Failed Cases:
Input: [10, -1] | Expected: 1 | Got: -1
Input: [-10, -1000] | Expected: 10 | Got: -10
Input: [50, -99] | Expected: 1 | Got: -1

Hidden function:

```
def gcd_euclid(a: int, b: int) -> int:
"""Greatest common divisor (non-negative)."""
a, b = abs(a), abs(b)
while b != 0:
a, b = b, a % b
return a
```

Your guess:

```
def solution(a, b):
while b:
a, b = b, a % b
return a
```

{'max_allowed_turns': 10, 'total_turns_used': 8, 'total_turns_remaining': 2}

Parse error count: 0

Results

	-, clemscore	Functionigma Game, % Played	Functionigma Game, Quality Score	Functionigma Game, Quality Score (std)	all, Average % Played	all, Average Quality Score
command-a-03-2025-t0.0	55.56	100.0	55.56	52.70	100.0	55.56
gpt-3.5-turbo-t0.0	0.00	100.0	0.00	0.00	100.0	0.00
gpt-4.1-2025-04-14-t0.0	78.57	100.0	78.57	42.58	100.0	78.57
gpt-4o-2024-08-06-t0.0	0.00	100.0	0.00	0.00	100.0	0.00

Whats next

- Search for better prompts:
  - A good prompt should function for the “dumbest” model (at least the format of inputs should be followable)
  - More:
    - Models to test on
    - Scoring metrics
    - Problems
    - Refinement (but later)